

Low-Latency DASH Live Streaming Pipeline

CS 418 Project 3 - Final Presentation

Atakan Karataş

Department of Computer Engineering, Özyeğin University

May 30, 2026

Outline

1. **Motivation & Task**
2. **System Architecture**
3. **FFmpeg Source Modification**
4. **Core Validation**
5. **Segment Duration Experiment**
6. **Fragment Duration Experiment**
7. **Bonus: QR Latency Monitor**
8. **Conclusion**

Motivation & Task

Core Objective

Build a low-latency live streaming pipeline to achieve glass-to-glass latency of **under 5 seconds** using open-source tools.

- **Standard HTTP Streaming:** HLS/DASH introduce 6-30s latency due to segment buffering.
- **Low-Latency DASH (LL-DASH):** Solves this by utilizing HTTP/1.1 chunked transfer encoding.
- **Deliverables:** Working stream pipeline, parameter experiments, analysis of latency trade-offs, and final report.

System Architecture

The delivery pipeline consists of five connected components:

1. **OBS Studio:** Media source. Captures video and embeds high-precision wall-clock time overlay.
2. **RTMP Link:** Encoded media transmission from OBS to local packaging process.
3. **Modified FFmpeg:** Packages incoming RTMP stream into low-latency DASH chunks/segments on local disk.
4. **NGINX & node-gpac-dash:** Serves the browser player page and delivers media chunks progressively.
5. **dash.js Player:** Browser playback engine with live synchronization and catchup logic.

FFmpeg Source Modification

- **The Problem:** Standard FFmpeg writes segments with a temporary `.tmp` extension and renames them only when complete. This blocks the server from progressive reads.
- **The Solution (Appendix A):** Modify `libavformat/dashenc.c` to write directly to the final file name:

Source Change in dashenc.c

Original: `use_rename ? "%s.tmp" : "%s"`

Modified: `use_rename ? "%s" : "%s"`

- **Verification:** Compiled from source with configure flags: `--enable-demuxer=dash --enable-libxml2`. Verified via `ffmpeg -h muxer=dash`.

Core Validation

- **Chunked Delivery:** Verified via browser Network tab showing Transfer-Encoding: chunked for .m4s segment requests.
- **Stabilizing Encoder Lag:** OBS encoding resolution and bitrate reduced to prevent CPU/encoder lag:
 - Output Resolution: **852x480**
 - Output Bitrate: **1200 kbps** (30 FPS)
- **Baseline Result:** 1-second segment profile reached stable **~4.0 seconds** of glass-to-glass latency without stalls.

Segment Duration Experiment

Varying segment duration (`seg_duration`) and keyframe interval (`keyint`) at 30 FPS:

Seg. Duration (s)	Keyint	Measured Latency (ms)	Playback Stability
1	30	4000	Stable (Target Met)
2	60	5500	Stable (Target Missed)
3	90	5500	Stable (Target Missed)
4	120	5500	Stable (Target Missed)
5	150	7000	Repeated stalls
6	180	7500	Stable but slow

- **Finding:** Latency scales directly with segment duration. Only the 1-second segment profile remained below the 5s threshold.

Fragment Duration Experiment

Testing fragment duration (`frag_duration`) with fixed 4s segments and `keyint=120` :

Fragment Duration (s)	Latency (ms)	Playback Stability
0.033	5000	Stable
0.066	5500	Stable
0.100	5500	Stable
0.500	6000	Stable
1.000	6000	Minor playback waiting / catchup
2.000	6000	Gap Controller adjustments
4.000	9000	Severe buffer depletion / stalls

- **Finding:** Fragment size alone does not bypass segment-level buffer delays. Large fragments (1s - 4s) cause playback stalls.

Bonus: QR Latency Monitor

- **Overlay:** Generates QR code containing local clock timestamp.
 - Payload format: `cs418-project3-ts=<ISO timestamp>`
- **Player Integration:**
 - Embedded a **QR-Based Latency Monitor** panel.
 - Samples video frames to an offscreen canvas.
 - Decodes payload in real-time using the `jsQR` library.
 - Subtracts embedded time from current browser system clock.
- **Advantage:** Enables automated, sub-second precision latency monitoring without manual stopwatch screenshots.

Conclusion

1. **Target Met:** Low-latency live streaming achieved at **4.0 seconds** of adjusted latency.
2. **End-to-End Dependency:** Low latency is a system-wide attribute. Buffering in NGINX, chunk handling in node-gpac-dash, and player properties in dash.js are as critical as the encoder settings.
3. **Chunked delivery is essential:** Prevents standard segment compilation delays.
4. **Validation Integrity:** Verified locally through reproducible scripts and real metrics.

Thank You

Questions?

Q&A Session